

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Mock Interview

1. RL Agent Learning Optimal Behavior

- **AIM**

To develop a Reinforcement Learning agent that learns optimal behavior by interacting with an environment and maximizing cumulative rewards.

- **ABSTRACT**

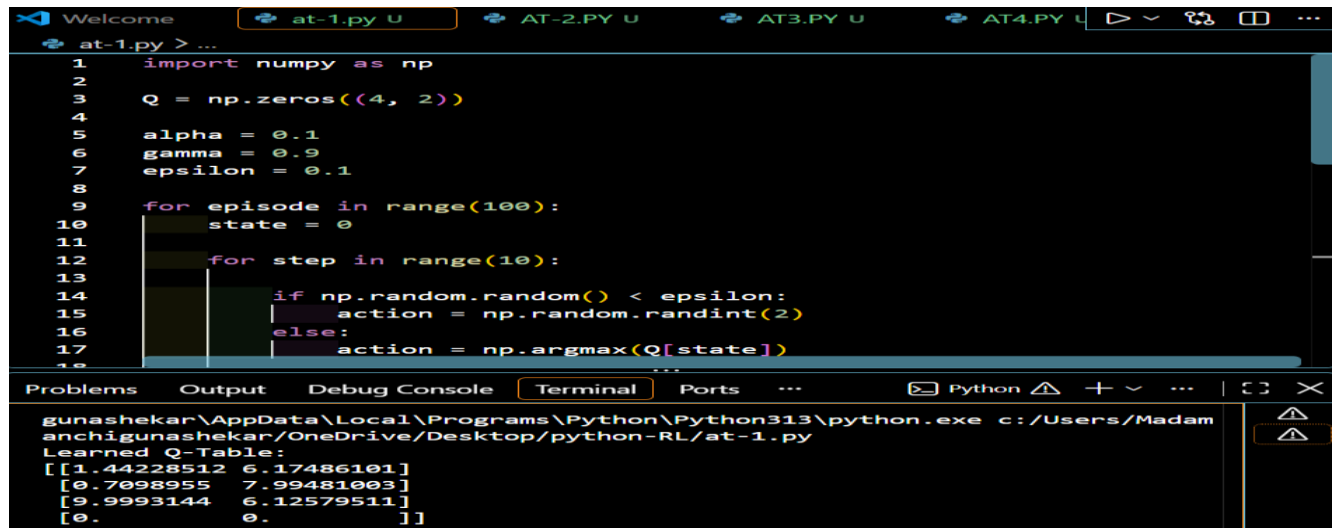
Reinforcement Learning is a machine learning technique in which an agent learns through trial and error. The agent observes the current state, selects an action, receives a reward, and moves to a new state. By repeating this process, the agent learns which actions provide maximum rewards.

- **ALGORITHM**

1. Initialize the Q-table with zeros.
2. Observe the current state.
3. Select an action using ϵ -greedy strategy.
4. Perform the selected action.
5. Receive reward and next state.
6. Update the Q-value.

7. Move to the next state.
8. Repeat until the episode ends.
9. Continue training for multiple episodes.
10. Select the action with the highest Q-value.

- **OUTPUT:-**



The screenshot shows a Python IDE with a file named 'at-1.py' open. The code implements a Q-learning agent with 4 states and 2 actions. It runs for 100 episodes, each with 10 steps. The output in the terminal window shows the learned Q-table after training:

```

gunashekar\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Madam
anchigunashekar/OneDrive/Desktop/python-RL/at-1.py
Learned Q-Table:
[[1.44228512 6.17486101]
 [0.7098955  7.99481003]
 [9.9993144  6.12579511]
 [0.         0.         ]]

```

- **CONCLUSION**

The RL agent successfully learns the best actions through repeated interaction with the environment. The Q-values improve over time, allowing the agent to select actions that maximize future rewards.

2. MDP for Real-Time Delivery Optimization

- **AIM**

To formulate a real-time delivery optimization problem using a Markov Decision Process and identify optimal delivery actions.

- **ABSTRACT**

A delivery system can be represented as an MDP where the vehicle location, traffic, battery level, and pending deliveries represent the state. The agent selects actions such as choosing a route or delivery location. Rewards are provided for fast deliveries and penalties are given for delays and high energy consumption.

- **ALGORITHM**

1. Define the states of the delivery system.
2. Define possible actions.
3. Define the reward for each action.

4. Define state transitions.
5. Initialize the value table.
6. Select an action for the current state.
7. Calculate the expected future reward.
8. Update the value.
9. Repeat until the values converge.
10. Select the action with maximum value.

OUTPUT:-

The screenshot shows a code editor with a file named 'AT-2.PY'. The code defines states, actions, and rewards for a delivery optimization problem. The terminal output shows the execution results for three states: Warehouse, Location_A, and Location_B, each with a best action and a reward.

```

1 states = ["Warehouse", "Location_A", "Location_B", "Customer"]
2
3 actions = {
4     "Warehouse": ["A", "B"],
5     "Location_A": ["Customer"],
6     "Location_B": ["Customer"]
7 }
8
9 rewards = {
10    ("Warehouse", "A"): 5,
11    ("Warehouse", "B"): 3,
12    ("Location_A", "Customer"): 10,
13    ("Location_B", "Customer"): 7
14 }
15
16 print("Delivery Optimization")
17
18 for state in states:

```

Terminal (Ctrl+`):

```

Delivery Optimization
State: Warehouse
Best Action: A
State: Location_A
Best Action: Customer
State: Location_B
Best Action: Customer

```

CONCLUSION:-

The delivery problem can be effectively represented using an MDP. By defining states, actions, rewards, and transitions, an RL agent can learn efficient delivery decisions while reducing travel time and operational cost.

3. Model-Free vs Model-Based RL

- **AIM**

To implement and compare Model-Free and Model-Based Reinforcement Learning approaches.

- **ABSTRACT**

Model-Free RL learns directly from experience without explicitly learning the environment model.

Model-Based RL uses an environment model to predict future states and rewards before selecting an action. Model-Free methods require more experience, whereas Model-Based methods can use planning to reduce the number of real interactions.

- **ALGORITHM**

Model-Free:

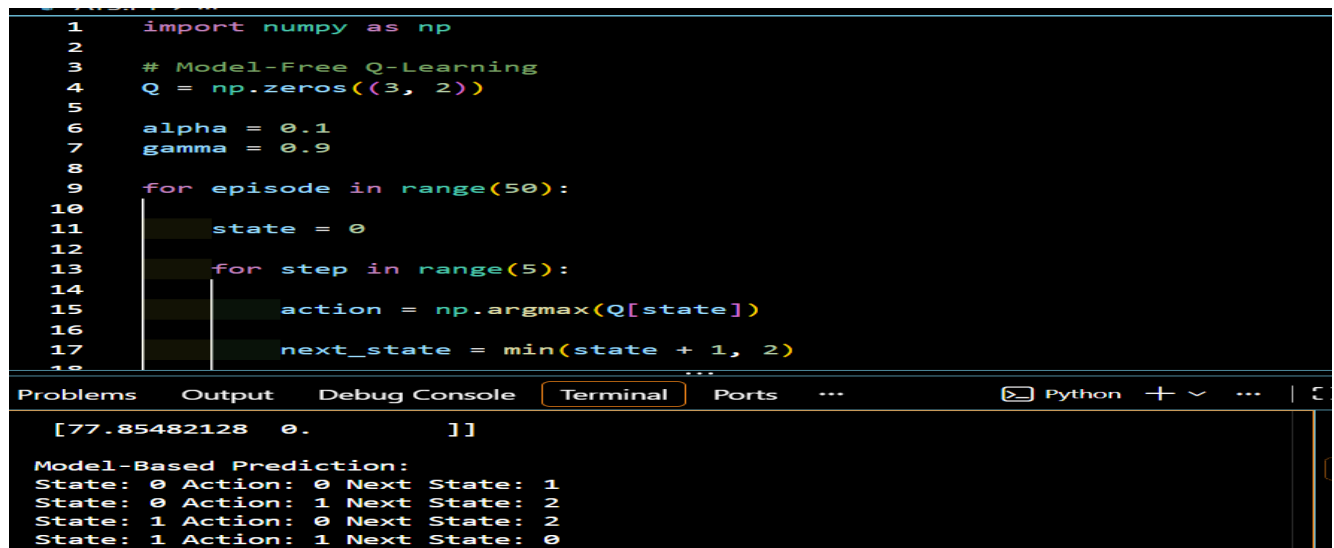
1. Initialize Q-values.
2. Observe the state.
3. Select an action.
4. Receive reward.
5. Observe the next state.

6. Update Q-value.
7. Repeat.

Model-Based:

1. Observe the environment.
2. Build or estimate a transition model.
3. Predict possible next states.
4. Calculate expected rewards.
5. Select the best predicted action.
6. Execute the action.
7. Update the model using new experience.

OUTPUT:-



```

1  import numpy as np
2
3  # Model-Free Q-Learning
4  Q = np.zeros((3, 2))
5
6  alpha = 0.1
7  gamma = 0.9
8
9  for episode in range(50):
10     state = 0
11
12     for step in range(5):
13
14         action = np.argmax(Q[state])
15
16         next_state = min(state + 1, 2)
17
18

```

Problems Output Debug Console **Terminal** Ports ... Python + v ... |

```

[77.85482128  0.         ]]
Model-Based Prediction:
State: 0 Action: 0 Next State: 1
State: 0 Action: 1 Next State: 2
State: 1 Action: 0 Next State: 2
State: 1 Action: 1 Next State: 0

```

CONCLUSION:-

Model-Free RL learns directly from experience, while Model-Based RL uses predictions of the environment for planning. Model-Free methods are simpler, whereas Model-Based methods can be more sample-efficient when the environment model is accurate.

4. Reward Sparsity and Hyperparameter Tuning

AIM

To improve an RL-based recommendation system by handling sparse rewards and tuning important hyperparameters.

ABSTRACT

In a recommendation system, rewards may be sparse because users do not interact with every recommended item. This makes learning difficult. Reward shaping can provide intermediate rewards

for actions such as clicking, viewing, or liking an item. Hyperparameters such as learning rate, discount factor, and exploration rate can then be tuned to improve performance.

ALGORITHM

1. Initialize the recommendation agent.
2. Observe the user's current state.
3. Recommend an item.
4. Record user interaction.
5. Assign a reward.
6. Use intermediate rewards for clicks and views.
7. Update the Q-value.
8. Tune learning rate and discount factor.
9. Adjust exploration rate.
10. Compare cumulative rewards and select the best configuration.

OUTPUT:-

```
8
9   for episode in range(100):
10
11       state = np.random.randint(3)
12
13       if np.random.random() < epsilon:
14           action = np.random.randint(3)
15       else:
16           action = np.argmax(Q[state])
17
18       # User interaction
19       interaction = np.random.choice(
20           ["view", "click", "like"])
21
22
23       if interaction == "view":
24           reward = 1
25       elif interaction == "click":...
```

Problems Output Debug Console Terminal Ports ... Python + v ...

C:\Users\Madamanchigunashekar\OneDrive\Desktop\python-RL>C:\Users\Madamanchigunashekar\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Madamanchigunashekar/OneDrive/Desktop/python-RL/AT3.PY

Recommendation Q-Table:

```
[[1.56749057 7.78877173 1.65024776]
 [7.84195058 1.12375574 1.11499492]
 [8.11047436 0.36131061 0.62173824]]
```

CONCLUSION:-

Reward shaping provides useful feedback even when final rewards are rare. Proper tuning of learning rate, discount factor, and exploration rate helps the recommendation agent learn more effectively and improve user engagement.

5. RL-Based Game-Playing AI

AIM

To develop an RL-based game-playing AI that learns optimal actions while maintaining stable training.

ABSTRACT

A game-playing AI can use Reinforcement Learning to learn by playing games repeatedly. The agent observes the game state, selects an action, receives a reward, and learns from the experience. Training stability can be improved using exploration control, experience replay, suitable learning rates, and periodic evaluation.

ALGORITHM

1. Initialize the game environment.
2. Initialize the RL agent.
3. Observe the game state.
4. Select an action using ϵ -greedy strategy.
5. Execute the action.
6. Receive reward and next state.
7. Store the experience.
8. Train the agent using stored experiences.
9. Reduce ϵ gradually to decrease random actions.
10. Repeat for multiple episodes.
11. Evaluate the trained agent using win rate and average reward.

OUTPUT:-

```
2
3  Q = np.zeros((5, 2))
4
5  alpha = 0.1
6  gamma = 0.9
7  epsilon = 1.0
8
9  for episode in range(100):
10
11      state = 0
12      done = False
13
14      while not done:
15
16          # Exploration and exploitation
17          if np.random.random() < epsilon:
18              action = np.random.randint(2)
```

blems Output Debug Console Terminal Ports ... Python + v ...

anchigunashekar/OneDrive/Desktop/python-RL/AT3.PY

Trained Game AI Q-Table:

```
[[4.48495968 0.15683732]
 [1.71572253 6.16816896]
 [7.99308406 3.76934758]
 [6.12579511 9.9993144 ]
 [0.         0.         ]]
```

CONCLUSION

The game-playing AI learns through repeated interaction with the game environment. ϵ -greedy exploration allows the agent to discover new actions, while gradual reduction of ϵ helps it exploit learned strategies. Stable training results in improved game performance and higher cumulative rewards.

Code of Conduct:

*I E TEJAVARDHAN REDDY (192425297) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E TEJAVARDHAN REDDY

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately